

外部函数调用

Julia基础语法

苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

目录

1. Julia与C/C++互调用

2. Julia与Python互调用

苏州同元软控信息技术有限公司
版权所有，侵权必究。

1.1 Julia与C/C++互调用

■ Julia调用C/C++

在数值计算领域，存在很多用C或Fortran写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

可以使用 `ccall` 来生成一个对C/C++库函数的调用。`ccall`定义如下：

```
ccall(function_name, library), returntype, (argtype1, ...), argvalue1, ...)  
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)  
ccall(function_pointer, returntype, (argtype1, ...), argvalue1, ...)
```

注：`ccall`是Julia内置库Base的关键字，无需加载额外包

例如：

```
ccall(:clock, Int32, ())
```

调用C库里面的clock函数，空参，返回值类型为Int32

1.1 Julia与C/C++互调用

■ Julia调用C/C++

对于C标准库中的函数，Julia可以直接调用。

例如，C标准库中的getenv函数声明：char* getenv (const char* name);

示例：

```
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)
```

函数名 返回值类型 参数类型 实参

```
julia> path = ccall(:getenv, Cstring, (Cstring,), "PATH") #注：函数名前面加冒号表示symbol类型
Cstring(0x0000000028c4f68)
```

```
julia> unsafe_string(path) #Copy a string from the address of a C-style, 表示从C地址中取字符串值
"C:\\Program Files (x86)\\VMware\\VMware
Workstation\\bin\\;C:\\Windows\\system32;C:\\Windows;C:\\Windows\\System32\\Wbem;C:\\Windows\\System32\\Window
sPowerShell\\v1.0\\;C:\\Windows\\System32\\OpenSSH\\;C:\\Program Files (x86)\\NVIDIA
Corporation\\PhysX\\Common;C:\\Program Files\\TortoiseSVN\\bin;C:\\ProgramData\\chocolatey\\bin;D:\\Program
Files\\Polyspace\\R2020b\\runtime\\win64;D:\\Program Files\\Polyspace\\R2020b\\bin;D:\\P" ... 788 bytes ...
"ingw64\\bin;C:\\Program Files\\CMake\\bin;D:\\Program Files\\MWorks.Syslab 2022\\Syslab\\bin;D:\\Program
Files\\MWorks.Syslab 2022\\Tools\\PortableGit\\cmd;D:\\Program Files\\MWorks.Syslab
2022\\Tools\\PortableGit\\usr\\bin;D:\\Users\\TR\\AppData\\Local\\Programs\\Microsoft VS Code\\bin;C:\\Program
Files\\MWorks.Syslab 2022\\Syslab\\bin;C:\\Program Files\\MWorks.Syslab
2022\\Tools\\PortableGit\\cmd;C:\\Program Files\\MWorks.Syslab 2022\\Tools\\PortableGit\\usr\\bin"
```



1.1 Julia与C/C++互调用

■ Julia调用C/C++

还可以使用另外一种方法 @ccall 来调用C函数，更加简洁清晰。和上一种方法使用等价

比如：ccall(:clock, Int32, ())等价于@ccall clock()::Int32

@ccall 的定义如下：

```
@ccall library.function_name(argvalue1::argtype1, ...)::returntype
@ccall function_name(argvalue1::argtype1, ...)::returntype
@ccall $function_pointer(argvalue1::argtype1, ...)::returntype
```

例如：

函数名 实参 参数类型 返回值类型

```
julia> path = @ccall getenv("PATH"::Cstring)::Cstring
Cstring(0x0000000028c4f68)
```

```
julia> unsafe_string(path)
```

1.1 Julia与C/C++互调用

■ Julia调用C++动态库

除了可以直接调用C标准库外，还可以调用用户自己开发的C++动态库。

假设自定义C++动态库路径为：f:/Syslab/Julia-C/ArrayMaker/x64/Release/ArrayMaker.dll

```
julia> lib = joinpath(@__DIR__, "ArrayMaker", "x64", "Release", "ArrayMaker") #获取动态库路径  
"f:\\Syslab\\Julia-C\\ArrayMaker\\x64\\Release\\ArrayMaker"
```

```
julia> c = @ccall lib.GetSum(2::Cdouble, 3::Cdouble)::Cdouble  
5.0
```

库路径

与调用C标准库方法一样

注意，通过这种方式调用C++动态库函数，存在一个弊端：即动态库被加载后不会释放，直到julia退出。

1.1 Julia与C/C++互调用

■ Julia调用C/C++

除了可以直接调用C标准库外，还可以调用用户开发的C++动态库。

通过 **Libdl** 来实现加载和卸载C/C++动态库。libdl函数库为Julia内置函数库，无需手动安装

```
using Libdl

# 加载库
lib_path = joinpath(@__DIR__, "ArrayMaker", "x64", "Release", "ArrayMaker")
lib = Libdl.dlopen(lib_path)

# 获取调用函数的符号
GetSum = Libdl.dlsym(lib, :GetSum)

# 调用函数
c = @ccall $GetSum(2::Cdouble, 3::Cdouble)::Cdouble

...

# 关闭dll
Libdl.dlclose(lib)
```

库路径

与调用C标准库方法一样

使用的函数帮助文档链接: <https://juliacn.gitlab.io/JuliaZH.jl/stdlib/Libdl.html>

1.1 Julia与C/C++互调用

Julia调用C/C++

目前，基本的C/C++值类型可以转换为Julia类型，以C为前缀。标准Julia别名与Julia基本类型没有区别

C 类型	Fortran 类型	标准 Julia 别名	Julia 基本类型
unsigned char	CHARACTER	Cuchar	UInt8
bool (<code>_Bool</code> in C99+)		Cuchar	UInt8
short	INTEGER*2, LOGICAL*2	Cshort	Int16
unsigned short		Cushort	UInt16
int, BOOL (C, typical)	INTEGER*4, LOGICAL*4	Cint	Int32
unsigned int		Cuint	UInt32
long long	INTEGER*8, LOGICAL*8	Clonglong	Int64
unsigned long long		Culonglong	UInt64
intmax_t		Cintmax_t	Int64
uintmax_t		Cuintmax_t	UInt64
float	REAL*4i	Cfloat	Float32
double	REAL*8	Cdouble	Float64
complex float	COMPLEX*8	ComplexF32	Complex{Float32}
complex double	COMPLEX*16	ComplexF64	Complex{Float64}
ptrdiff_t		Cptrdiff_t	Int
ssize_t		Cssize_t	Int
size_t		Csize_t	UInt

C 类型	Fortran 类型	标准 Julia 别名	Julia 基本类型
void			Cvoid
void and <code>[[noreturn]]</code> or <code>_Noreturn</code>			Union{}
void*			Ptr{Cvoid} (或类似的 Ref{Cvoid})
T* (where T represents an appropriately defined type)			Ref{T} (只有当 T 是 isbits 类型时, T 才可以安全地转变)
char* (or <code>char[]</code> , e.g. a string)	CHARACTER*N		Cstring if NUL-terminated, or Ptr{UInt8} if not
char** (or <code>*char[]</code>)			Ptr{Ptr{UInt8}}
julia_value_t* (any Julia Type)			Any
julia_value_t* const* (一个 Julia 值的引用)			Ref{Any} (常量, 因为转变需要写屏障, 不可能正确插入)
va_arg			Not supported
... (variadic function specification)			T... (其中 T 是上述类型之一, 当使用 <code>ccall</code> 函数时)
... (variadic function specification)			; <code>va_arg1::T</code> , <code>va_arg2::S</code> 等 (仅支持 <code>@ccall</code> 宏)

上面标注红框为常用类型



1.1 Julia与C/C++互调用

■ Julia调用C/C++的一个完整示例

① Julia代码:

```
using Libdl

# 加载dll
lib_path = joinpath(@__DIR__, "ArrayMaker", "x64", "Release", "ArrayMaker")
lib = Libdl.dlopen(lib_path)

# 获取符号
CreateObj = Libdl.dlsym(lib, :CreateObj)
DeleteObj = Libdl.dlsym(lib, :DeleteObj)
FillArray = Libdl.dlsym(lib, :FillArray)

# 创建对象指针
pobj = @ccall $CreateObj()::Ptr{Cvoid}

# 填充数组
len = 5
parr = @ccall $FillArray(pobj::Ptr{Cvoid}, len::Cint, 3.5::Cdouble)::Ptr{Cdouble}
arr = [unsafe_load(parr, i) for i = 1:len]
#=
5-element Vector{Float64}:
 3.5
 3.5
 3.5
 3.5
 3.5
=#

# 销毁对象
@ccall $DeleteObj(Ref(pobj)::Ptr{Ptr{Cvoid}})::Cvoid
pobj = C_NULL

# 关闭dll
Libdl.dlclose(lib)
```

② C/C++头文件代码:

```
#ifndef ARRAYMAKER_H
#define ARRAYMAKER_H

#include "ArrayMaker_global.h"

struct ArrayMaker {
    int nNumber;
    double* pArray;
};

extern "C" ARRAYMAKER_EXPORT ArrayMaker* CreateObj();
extern "C" ARRAYMAKER_EXPORT void DeleteObj(ArrayMaker ** ppobj);

extern "C" ARRAYMAKER_EXPORT double* FillArray(ArrayMaker* pobj, int num,
double value);

#endif
```

备注:

CreateObj: 创建数组构造器对象;

DeleteObj: 销毁数组构造器对象;

FillArray: 填充数组, 输入参数为数组长度、数组元素值。初始时, 所有数据元素值都为value。

1.1 Julia与C/C++互调用

■ Julia调用C/C++的一个完整示例

③ C/C++源文件代码:

```
#include "ArrayMaker.h"
#include <math.h>

using namespace std;

ArrayMaker* CreateObj()
{
    auto p = new ArrayMaker;
    p->nNumber = 0;
    p->pArray = nullptr;
    return p;
}

void DeleteObj(ArrayMaker** ppobj)
{
    if (ppobj == nullptr) {
        return;
    }

    auto& pobj = *ppobj;
    if (pobj != nullptr) {

        //删除数据
        if (pobj->pArray != nullptr) {
            delete[] pobj->pArray;
            pobj->pArray = nullptr;
        }

        //删除本身
        delete pobj;
        pobj = nullptr;
    }
}
```

```
double* FillArray(ArrayMaker* pobj, int num, double value)
{
    double* data = nullptr;

    if (pobj != nullptr) {
        data = pobj->pArray;
        if (data != nullptr) {
            delete[] data;
        }

        data = new double[num];
        for (int i = 0; i < num; i++) {
            data[i] = value;
        }
        pobj->pArray = data;
        pobj->nNumber = num;
    }

    return data;
}
```

1.1 Julia与C/C++互调用

创建和C兼容的Julia函数指针(高阶)

```
@cfunction(callable, ReturnType, (ArgumentTypes...,)) -> Ptr{Cvoid}  
@cfunction($callable, ReturnType, (ArgumentTypes...,)) -> CFunction
```

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

1.1 Julia与C/C++互调用

创建和C兼容的Julia函数指针(高阶)

目前, Julia在调用C函数时, 还支持创建与C兼容的Julia函数指针, 并作为参数传递给C函数。

一个典型的

```
/* 标准C库函数qsort的定义
 * [in|out]base 集合
 * [in]num      集合元素个数
 * [in]size     每个元素的字节数
 * [in]compar   两个元素的比较函数
 */
void qsort (void* base, size_t num, size_t size,
            int (*compar)(const void*,const void*));
```

```
# Julia的比较函数
function mycompare(a, b)::Cint
    println("mycompare($a, $b)") #可以查看比较过程
    return (a < b) ? -1 : ((a > b) ? +1 : 0)
end

# 通过 @cfunction 获取函数指针
mycompare_c = @cfunction(mycompare, Cint, (Ref{Cdouble}, Ref{Cdouble}));

# 调用示例
A = [1.3, -2.7, 4.4, 3.1]
ccall(:qsort, Cvoid, (Ptr{Cdouble}, Csize_t, Csize_t, Ptr{Cvoid}),
      A, length(A), sizeof(eltype(A)), mycompare_c)

# 查看结果
A
#=
4-element Vector{Float64}:
-2.7
 1.3
 3.1
 4.4
=#
```

1.2 Julia与C/C++互调用

■ C/C++调用Julia

C/C++调用Julia, 本质上C/C++调用Julia动态库, C/C++工程的编译和运行都依赖Julia及Julia仓库。

C/C++调用Julia的配置方法, 详细请参见: <https://juliacn.gitlab.io/JuliaZH.jl/manual/embedding.html>。

下例为函数展示了使用c++启动julia, 并调用Julia的"print(sqrt(2.0))"语句

```
#include <julia.h>
JULIA_DEFINE_FAST_TLS // only define this once, in an executable (not in a shared
library) if you want fast code.

int main(int argc, char *argv[])
{
    /* required: setup the Julia context */
    jl_init();

    /* run Julia commands */
    jl_eval_string("print(sqrt(2.0))");

    /* strongly recommended: notify Julia that the
       program is about to terminate. this allows
       Julia time to cleanup pending write requests
       and run all finalizers
    */
    jl_atexit_hook(0);
    return 0;
}
```

目录

1. Julia与C/C++互调用

2. Julia与Python互调用

苏州同元软控信息技术有限公司
版权所有，侵权必究。

未经许可不得复制、传播或以其他方式使用；

2.1 Julia与Python互调用

■ Julia调用Python

在数值计算领域，存在很多用Python写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

Julia调用python扩展库中函数：**pyimport** 和 **@pyimport**

```
# 依赖PyCall
using PyCall

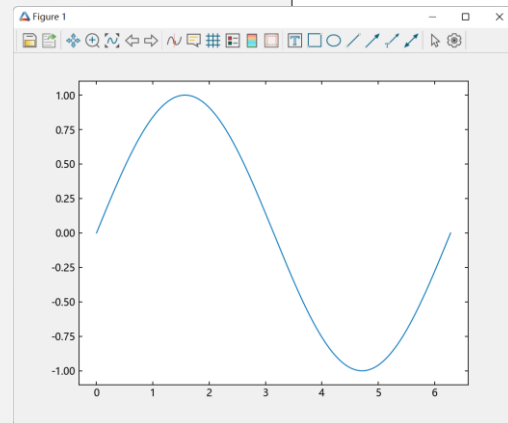
# 导入python库
math = pyimport("math")

v = math.sin(pi/2)
println("v = $v")
# v = 1.0
```

```
using PyCall
using TyPlot # 同元绘图库

# 导入python库
@pyimport numpy as np
x = np.linspace(0,2pi, 1000)
y = np.sin(x)

# 绘图
plot(x,y)
```



目前Syslab安装包已经提供了完备的Julia和Python。

若用户想要使用自己的Python环境，需要参考Syslab帮助文档“Syslab-外部语言接口-Python调用Julia”。

2.1 Julia与Python互调用

■ Julia调用Python

直接在Julia中嵌入python代码: **py**

```
using PyCall

# 直接嵌入python代码
py"""
str = "3 * 4 + 5"
a = compile(str, '', 'eval')
print("a =", eval(a))
"""

# a = 17
```

```
module MyModule
using PyCall

function __init__()

    py"""
    def hello(s):
        return "hello " + s
    """

end

# 直接调用python函数
hello(s) = py"hello"(s)

end # MyModule

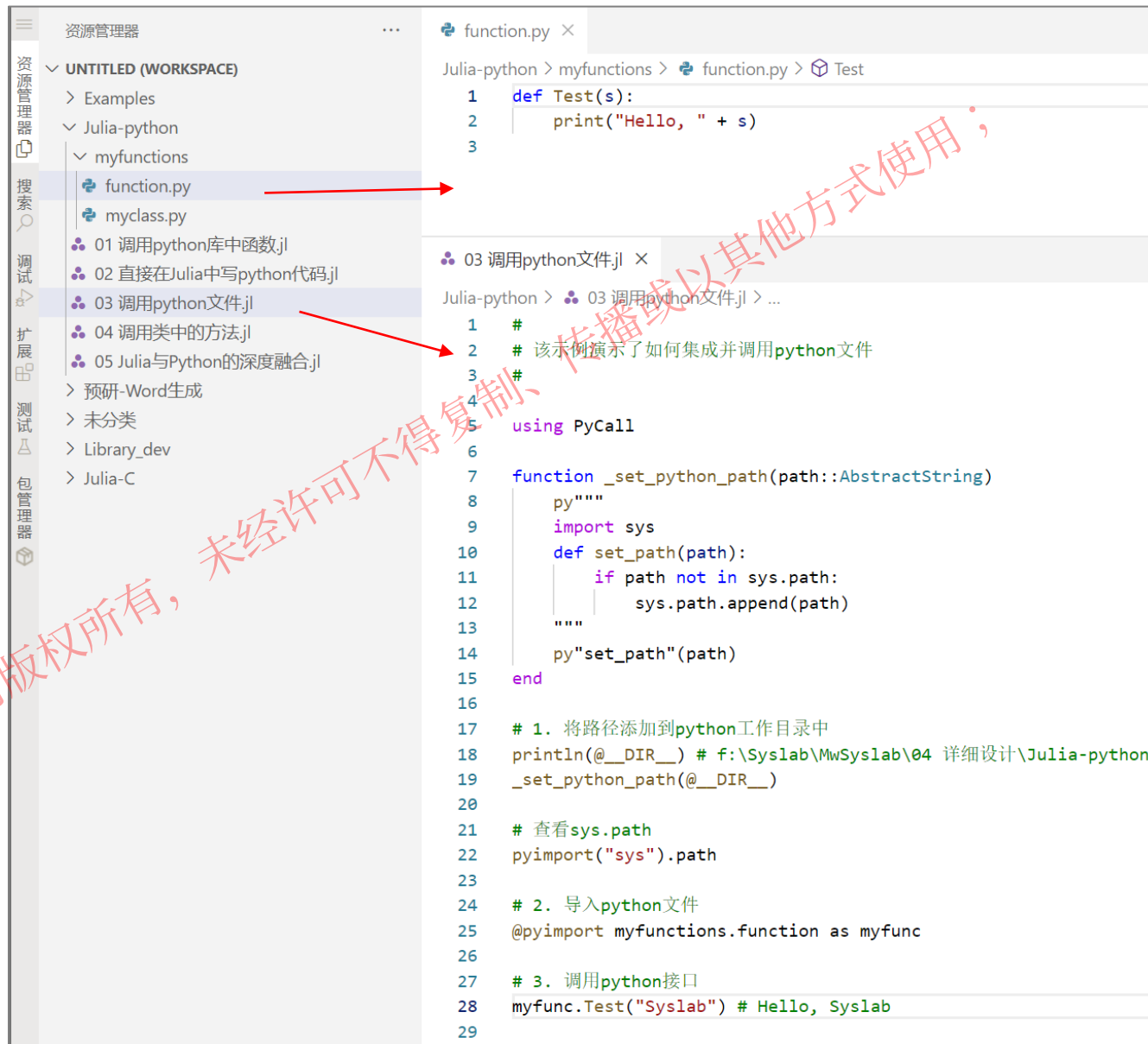
MyModule.hello("julia")
# "hello julia"
```


2.1 Julia与Python互调用

■ Julia调用Python

调用python代码文件:

- 将路径添加到python工作目录中
- 导入python文件
- 调用python接口



示例: 见Example示例: Julia-Python

2.1 Julia与Python互调用

Julia与Python深度融合

@pydef: 创建一个python类，其方法是用Julia实现的。

```
using PyCall

# @pydef: 创建一个python类，其方法是用julia实现的
P = pyimport("numpy.polynomial")

@pydef mutable struct Doubler <: P.Polynomial
    __init__(self, x = 10) = (self.x = x)
    my_method(self, arg1::Number) = arg1 + 20
    x2.get(self) = self.x * 2
    x2.set!(self, new_val) = (self.x = new_val / 2)
end

print(Doubler().x2) # 20
```

```
using PyCall

# python代码，与左边代码等价
py"""
import numpy.polynomial
class Doubler(numpy.polynomial.Polynomial):
    def __init__(self, x=10):
        self.x = x
    def my_method(self, arg1): return arg1 + 20
    @property
    def x2(self): return self.x * 2
    @x2.setter
    def x2(self, new_val):
        self.x = new_val / 2
print(Doubler().x2) # 20
"""
```



2.1 Julia与Python互调用

■ Julia调用Python

调用python类中的方法:

- 将路径添加到python工作目录中
- 导入python文件
- 获取类对象
- 调用python类中的方法

```
resource manager
└─ UNTITLED (WORKSPACE)
   └─ Examples
      └─ Julia-python
         └─ myfunctions
            ├── function.py
            └── myclass.py
               ├── 01 调用python库中函数.jl
               ├── 02 直接在Julia中写python代码.jl
               ├── 03 调用python文件.jl
               └── 04 调用类中的方法.jl
                  └── 05 Julia与Python的深度融合.jl
                     └─ 预研-Word生成
                        └─ 未分类
                           └─ Library_dev
                              └─ Julia-C
```

```
function.py
1 import numpy.polynomial
2
3 class MyNet(numpy.polynomial.Polynomial):
4     def __init__(self, x=10):
5         self.x = x
6     def add(self, a):
7         return self.x+a
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
```

```
myclass.py
10 using PyCall
11
12 function _set_python_path(path::AbstractString)
13     py"""
14     import sys
15     def set_path(path):
16         if path not in sys.path:
17             sys.path.append(path)
18     """
19     py"set_path"(path)
20 end
21
22 # 将路径添加到 python 工作目录中
23 println(@__DIR__) # f:\Syslab\MwSyslab\04 详细设计\Julia-python
24 _set_python_path(@__DIR__)
25
26 # 导入python文件
27 @pyimport myfunctions.myclass as myclass
28
29 # 调用python类中的方法
30 my_net = myclass.MyNet()
31 my_net.add(20) # 30
32
```

示例: 见Example示例: Julia-Python

2.2 Julia与Python互调用

■ Python调用Julia

Julia通过PyCall可以很方便地调用Python代码。反过来，Python通过**tjc_common**、**tyjuliacall**、**julia-numpy**等扩展包来实现对Julia的调用（注，上述三个扩展包都是同元公司开发的）。

其中，tjc_common 主要是利用JuliaSysimage.dll(或.so) 文件来加速Julia包导入，[tyjuliacall](#) 主要用来实现Python对Julia代码的调用，julia-numpy 作为tyjuliacall的底层支撑。

■ 示例1-内嵌Julia代码

- 启动Syslab，新建Python文件
- 写入Python代码(如右图)
- 运行得到结果

```
from tjc_common import *
from tyjuliacall import Main
from tyjuliacall import JuliaEvaluator

# 调用Julia代码
JuliaEvaluator[
    """mutable struct S
        x :: Int
        y :: Int
    end"""
]
S = JuliaEvaluator["S"]
s = S(1, 2)
print(s.x)
print(s.y)

# 修改字段
s.x = 10
s.y = 5
print(s.x)
print(s.y)
```

```
输出 调试控制台 终端 + 3: Python (Examples)
PS C:\Program Files\MWORKS\Syslab 2023a\Examples> & C:/Users/Public/TongYuan/.julia/miniforge3/python.exe "c:/Program Files/MWORKS/Syslab 2023a/Examples/07 Interfaces/PythonCallJulia/test-struct.py"

导入tyjuliacall: 6.02 s
1
2
10
5
PS C:\Program Files\MWORKS\Syslab 2023a\Examples>
```

2.2 Julia与Python互调用

■ 示例2-调用Julia库函数

- 启动Syslab, 新建Python文件
- 写入Python代码(如右图)
- 运行得到结果

medfilt1是同元信号处理工具箱函数

plot是同元图形工具箱函数

```
from time import time
from tjc_common import *

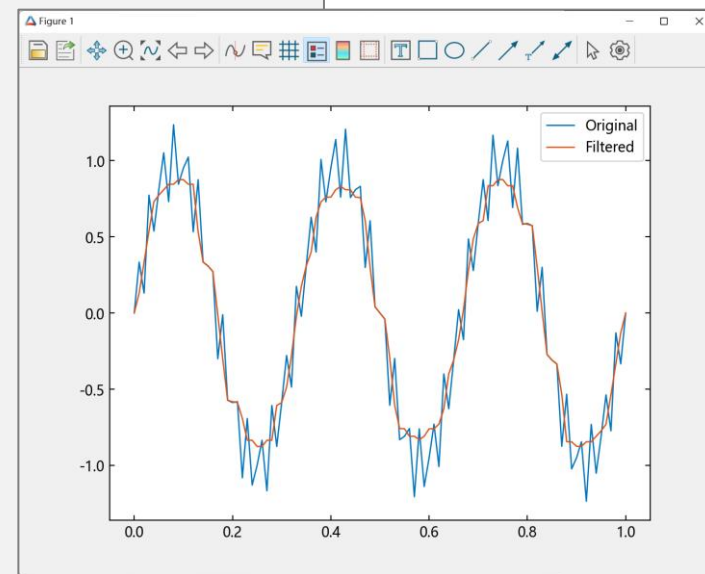
_t0 = time.time()
from tyjuliacall import TySignalProcessing as sp # NOQA: E402
from tyjuliacall import TyPlot as tp # NOQA: E402
print(f"导入同元库耗时: {get_timespan(_t0):.2f} s")

import numpy as np

fs = 100
t = np.arange(fs + 1) / fs
print(t)
x = np.sin(2 * np.pi * t*3) + 0.25*np.sin(2 * np.pi * t*40)

# 调用信号库函数
_t0 = time.time()
y = sp.medfilt1(x, 9)
print(f"调用函数耗时: {get_timespan(_t0):.2f} s")

# 调用图形库函数
tp.plot(t, x, t, y)
tp.legend(np.asarray(["Original", "Filtered"]))
tp.plt.show()
```



建立知识规范，营造协同生态

积累工业模型，发展可控平台

融入中国创新，打造先进软件

请各位专家指正！

苏州同元软控信息技术有限公司
版权所有，侵权必究。

未经许可，不得复制、传播或以其他方式使用；